Project plan: duplicate questions on Quora

02807 Computational Tools for Data Science, group of 4. Plan written 29 Oct 2026, hand-in Friday 4 Dec 2026, 23:59.

1. Project in one paragraph

We measure how many duplicate questions on Quora are missed by MinHash-LSH because they are paraphrases, and test whether adding sentence embeddings and graph clustering recovers them at a fraction of the cost of comparing all pairs. Our own MinHash-LSH produces lexical candidate pairs. Sentence embeddings with approximate nearest-neighbour search (HNSW) produce semantic candidate pairs. Both sets of pairs form a similarity graph, and clustering that graph groups duplicates transitively (if A matches B and B matches C, then A and C are in the same group).

Research question. How many labelled duplicate pairs does MinHash-LSH miss, how many of those can embeddings recover, and does graph clustering of the combined graph improve recall without merging unrelated questions?

Course topics. Similar items / LSH (essential: it produces the first set of candidates and is the baseline). Graph clustering with Louvain and modularity (essential: it turns pairs into groups and decides how far transitivity can be trusted).

New element. Sentence embeddings (bi-encoder, e.g. MiniLM) with approximate nearest-neighbour search using HNSW. The report has to explain how both work.

Own implementation. Vectorised MinHash with LSH banding, the candidate evaluation code, the graph construction and the modularity computation. Libraries are used for embeddings (sentence-transformers), HNSW (FAISS), and as a check for our MinHash (datasketch) and Louvain (NetworkX).

2. Data

Quora Question Pairs (Kaggle competition "Quora Question Pairs", file `train.csv`). Columns: `id, qid1, qid2, question1, question2, is_duplicate`. About 404k labelled pairs, about 537k distinct questions, roughly 37% of pairs labelled duplicate. Downloading needs a Kaggle login and accepting the competition rules. Backup copy: the GLUE QQP split on Hugging Face (same source, different split sizes). These figures and file names are from memory, so one person should confirm them on day one.

Preprocessing to document in the report

- Remove rows with missing or empty questions, and map identical texts under different ids to one question.
- Lower-case, normalise whitespace and punctuation. Decide whether to remove stop words (test it, do not assume).
- Very short questions give tiny shingle sets, so use character shingles or a minimum length for the LSH stage. Report how many questions are affected.
- Ground truth: build a graph from the `is_duplicate = 1` pairs. Its connected components are the true duplicate groups, and their transitive closure gives additional implied duplicate pairs. Decide in week 1 whether

evaluation uses the labelled pairs only, the closure, or both, and report both.

Evaluation caveat. The labels cover only a sample of all possible pairs. A candidate pair with no label may still be a true duplicate, so precision is underestimated. Plan: report precision on labelled candidates only, plus a manual check of 100 randomly chosen unlabelled candidates with high similarity.

3. Pipeline

1. **Clean and split.** One shared cleaned table of questions and a table of labelled pairs. Start with a 5 to 10 percent sample (as the TA advised), keeping all questions that appear in sampled pairs.
2. **MinHash-LSH (own).** Word and character shingles, MinHash signatures with vectorised NumPy hashing (the week 4 code uses a Python loop with mmh3, which is too slow at this size), LSH banding with b bands of r rows. Plot the empirical S-curve against the theory 1-(1-s^r)^b and choose b and r from the target Jaccard threshold. Validate against datasketch on a sample.
3. **Embeddings and HNSW.** Encode all questions with a small sentence-transformer, normalise, build a FAISS HNSW index and take the top 10 to 20 neighbours of each question above a cosine threshold. Validate the HNSW recall against exact brute-force search on a sample.
4. **Combined graph.** Nodes are questions. Edges come from LSH (estimated Jaccard above a threshold) and from embeddings (cosine above a threshold), with a weight and a source flag.
5. **Clustering.** Connected components versus Louvain on the combined graph. Modularity guides the choice of edge thresholds and Louvain resolution, and a chaining analysis shows where one wrong edge merges two groups.
6. **Evaluation.** Compare five systems: LSH only, embeddings only, union of candidates, union with connected components, union with Louvain. Metrics: recall, precision, F1 on pairs, number of candidates, runtime and memory against the all-pairs cost. Key analysis: bin the true duplicate pairs by their word-level Jaccard similarity and report how recall of each system changes by bin. This answers the research question directly.

4. Work split

Every role has one component that is demanding, so the load is equal. Everyone writes the code and the report section for their own part, and reviews one other person's section.

	Role	Tasks	Report section (target characters, no spaces)
P1	Data and evaluation	Download and cleaning, EDA, ground-truth graph and closure, evaluation harness (pair precision/recall/F1, cluster metrics, recall by Jaccard bin), manual precision check, notebook structure and README, final trimming of the report	Motivation, data and preprocessing, evaluation setup (about 2,400)
P2	MinHash-LSH	Vectorised MinHash, banding, shingle and parameter study (k, shingle size, b and r), S-curves, validation against datasketch, runtime against all-pairs	LSH method and results (about 2,000)
P3	Embeddings and HNSW	Embedding pipeline with caching to disk, FAISS HNSW, tuning of neighbours and threshold, validation against exact search, explanation of the new method	New element: embeddings and HNSW (about 2,000)
P4	Graph and clustering	Combined graph, Louvain and connected components, own modularity validated against NetworkX, threshold selection, chaining analysis, the combined experiments	Graph clustering, discussion and conclusion (about 2,400)

The remaining characters go to the summary at the top (about 800), written together in the last week. The target is at most 10,500 characters, under the 11,000 limit.

Shared interfaces, agreed in week 1 so that everyone can work in parallel:

- `questions.parquet`: `qid, text_raw, text_clean`
- `pairs.parquet`: `qid1, qid2, is_duplicate`
- candidate files: `qid_a, qid_b, score, source` (source is `lsh` or `emb`)
- embeddings: `embeddings.npy` plus the matching `qid` order
- one function `evaluate(candidates, truth)` that returns the metrics, owned by P1

5. Timeline

Dates	Goal
Thu 29 Oct to Sun 1 Nov	Confirm data access, create the shared notebook/repo, agree the interfaces above, P1 delivers the 5 to 10 percent sample. Each person reads the part of the lecture and the paper for their own component.
2 to 8 Nov	All four components working on the sample: cleaned data and evaluation harness (P1), MinHash and banding (P2), embeddings and HNSW (P3), graph builder on first candidates (P4).
9 to 15 Nov	Full-data runs. Parameter studies. Decision on 15 Nov: if embedding all 537k questions is too slow, fix a documented subset size and continue with it.
16 to 22 Nov	Combined experiments, clustering, the five-system comparison and recall-by-bin analysis. Manual precision check.
23 to 29 Nov	Error analysis (why LSH misses, where chaining fails), figures, first draft of each report section. Only if ahead of schedule: one harder extension (below).
30 Nov to Fri 4 Dec	Merge and cut the report to the character limit, run the notebook from scratch on a clean machine, write the README and the contribution outline. Submit by Thursday 3 Dec evening, leaving 4 Dec as buffer.

The project description gives 4 December at the top and 6 December later. This plan uses 4 December.

6. Risks

Risk	Mitigation
Embedding 537k questions is slow on CPU (our estimate: one to a few hours with several cores, minutes on a GPU)	Use a small model (MiniLM), cache the result to disk once, use a free Colab GPU, or fall back to a documented subset.
Pure-Python MinHash is too slow	Vectorised NumPy hashing, or Numba (week 2) for the hash loop.
Quora labels are incomplete and noisy, so precision looks low	Report both labelled-only precision and the manual sample, and explain the limit in the report.
Louvain chains unrelated questions into big groups	Raise edge thresholds, use resolution above 1, report chaining as a finding with examples.
Short questions have too few shingles	Character shingles, or handle questions below a length threshold with embeddings only.
Four people writing 4 pages	One person (P1) owns the final trimming, everyone checks the character count of their section weekly from week 4.

7. Optional harder extensions

Only after the main pipeline works, around 23 Nov, and only one of them: implement HNSW ourselves and benchmark it against FAISS, add multi-probe LSH, or fine-tune the embedding model on the labelled pairs. If none is done, the project is still complete.

8. Report outline (at most 11,000 characters, about 4 pages)

1. Summary of the whole project, 5 lines (about 800)
2. Problem and motivation (about 700)
3. Data and preprocessing (about 1,500)
4. Methods: MinHash-LSH, then embeddings and HNSW (the new element), then graph clustering, each with the reason it fits the data (about 4,000)
5. Evaluation setup and results, including the recall-by-bin figure (about 2,400)
6. Discussion: why LSH misses what it misses, where embeddings fail, what chaining does (about 1,000)
7. Conclusion (about 400)

The notebook is submitted as the appendix and has to run with clear instructions, plus the contribution outline for each member.